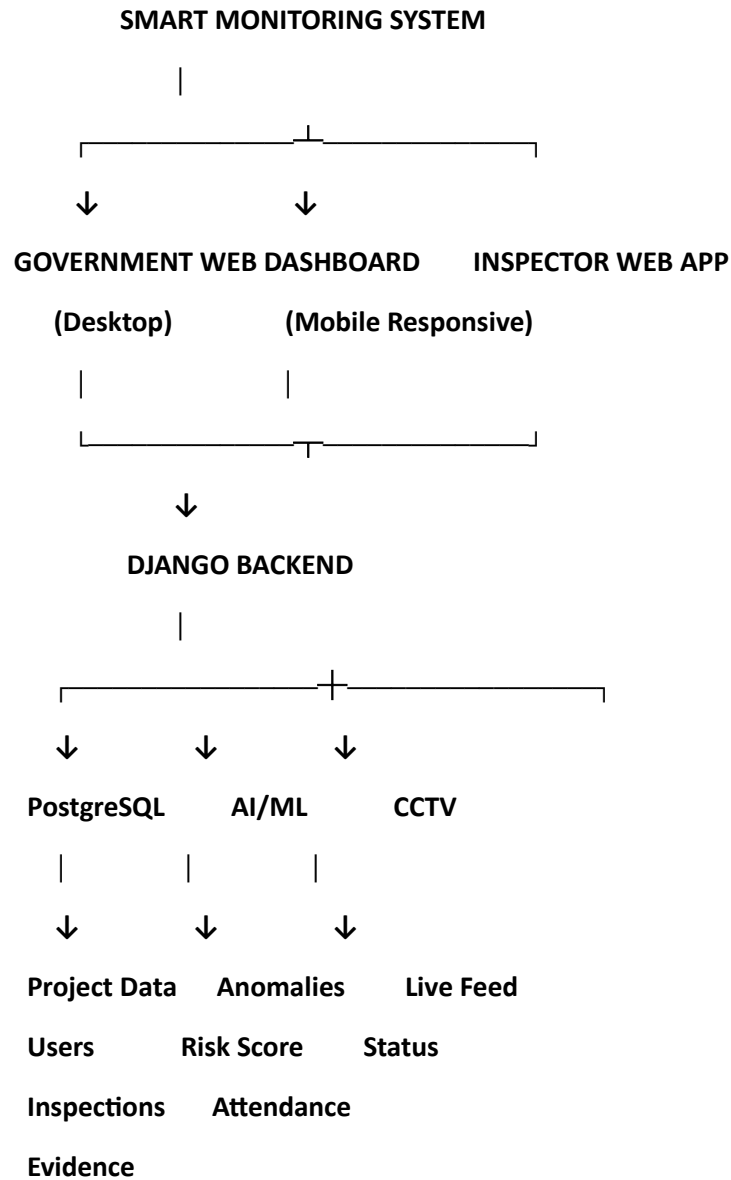


Complete Implementation Plan

Smart Real-Time Monitoring & Inspection System — PS 26095

The system will have three main parts:



The original problem asks for live CCTV, random VC, real-time monitoring, inspection, automated assignment, geo-tagged evidence, and AI-based anomaly/attendance analytics.

1. Technology Stack

We're intentionally removing React Native.

Component	Technology
Programming	Python + JavaScript

Component	Technology
Backend	Django
API	Django REST Framework
Database	PostgreSQL
Frontend	React.js
UI	HTML + CSS / Tailwind CSS
Maps	Leaflet + OpenStreetMap
GPS	Browser Geolocation API
CCTV	OpenCV
CCTV Streaming	WebRTC / HLS
AI/ML	Python + Scikit-learn
Computer Vision	OpenCV + YOLO
Video Conference	WebRTC
Real-time	Django Channels + WebSockets
Background Tasks	Celery
Message Broker	Redis
Notifications	Browser notifications initially
File Storage	Django Media
API Testing	Postman
Database Tool	pgAdmin
Code	VS Code
Version Control	Git + GitHub
Deployment	Render + Vercel

Don't install everything immediately.

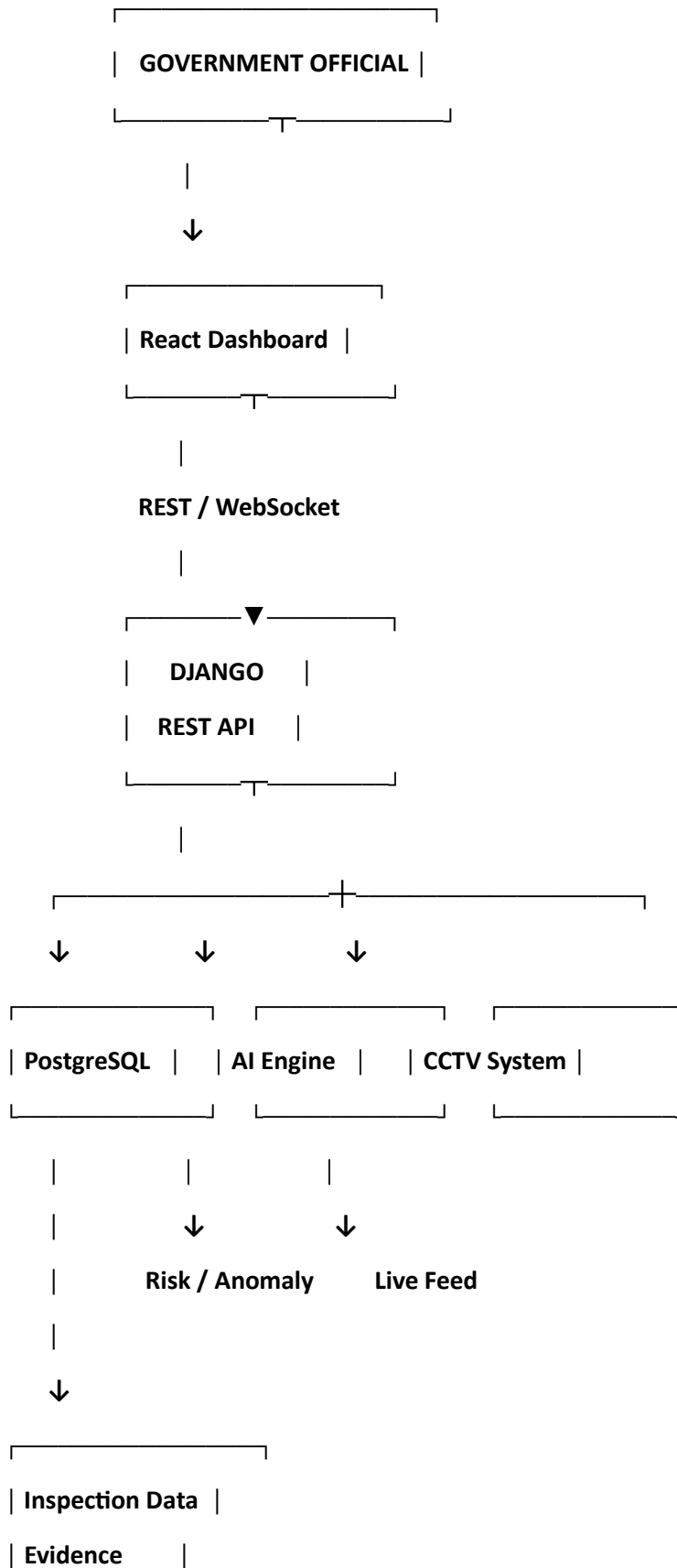
Start with:

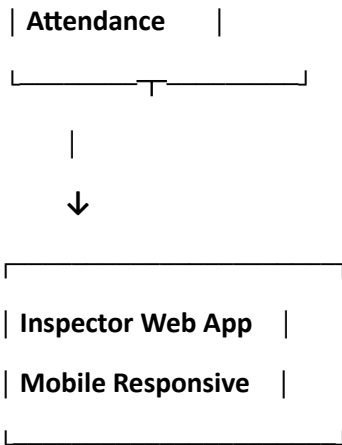
Python → Django → PostgreSQL → React → Git

Then add the advanced components one by one.

2. Overall Architecture

Your final system:





3. Django Project Structure

Create one Django project with multiple applications.

smart-monitoring/

```
|
|
| └─ backend/
|
| |
| | └─ manage.py
| |
| |
| | └─ config/
| |   └─ settings.py
| |   └─ urls.py
| |   └─ asgi.py
| |   └─ wsgi.py
| |
| |
| | └─ accounts/
| | └─ projects/
| | └─ inspections/
| | └─ assignments/
| | └─ attendance/
| | └─ cctv/
| | └─ evidence/
```

```
|   ├── analytics/
|   ├── notifications/
|   └── reports/
|
|   ├── frontend/
|   └── React application
|
|   ├── ai/
|   ├── anomaly_detection.py
|   ├── attendance_analysis.py
|   └── person_detection.py
|
|   ├── media/
|
|   ├── requirements.txt
└── README.md
```

4. Django Apps

Don't put everything inside models.py of one application.

Use:

accounts

Users and roles.

projects

NGOs, institutes and projects.

inspections

Inspection process.

assignments

Automatic inspection assignment.

attendance

Attendance records.

cctv

Camera information and streaming.

evidence

Photos/videos/documents.

analytics

AI and risk scoring.

notifications

Alerts and notifications.

reports

Inspection reports.

5. User Management

Create a custom Django User.

Roles:

ADMIN

OFFICIAL

INSPECTOR

NGO

STAFF

Database:

User

id

name

email

phone

password

role

is_active

created_at

6. Role Permissions

Admin
Everything.
Users
Projects
CCTV
Inspections
AI
Reports
Settings
Government Official
Dashboard
Projects
CCTV
AI Alerts
Inspections
Reports
Inspector
My Inspections
GPS
Checklist
Camera
Evidence
Submit Report
NGO
Own Project
Staff
Beneficiaries
Attendance
CCTV

7. Authentication

Use:

Django REST Framework + JWT

Package:

django-rest-framework-simplejwt

Flow:

React



Login



Django



JWT Token



React



Authenticated API requests

8. Project Management

Create a Project model.

Project

id

project_name

ngo_name

scheme

description

address

state

district

latitude

longitude

start_date

end_date

staff_count

beneficiary_count

status

created_at

Example:

ABC Skill Development Center

Mumbai

Beneficiaries: 150

Staff: 12

Status: Active

9. Project Map

Government dashboard should display all projects on a map.

Use:

Leaflet + OpenStreetMap

Markers:

 **Low Risk**

 **Medium Risk**

 **High Risk**

Clicking a marker:

ABC Foundation

Location: Mumbai

Beneficiaries: 150

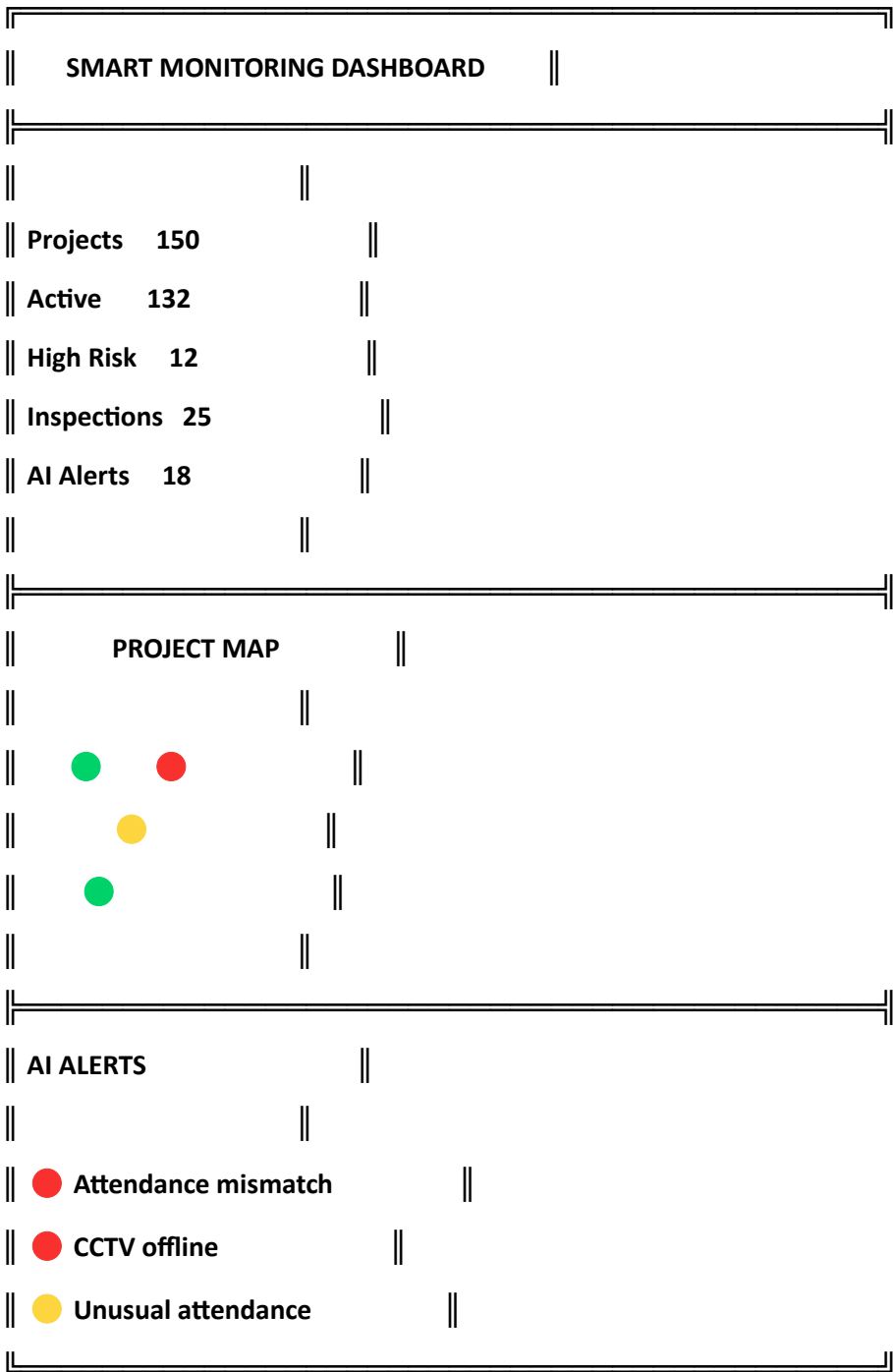
CCTV: ONLINE

Risk: HIGH

[VIEW PROJECT]

10. Dashboard

This should be your main presentation screen.



11. Inspector Interface

Instead of React Native, make a mobile-responsive React website.

When opened on a phone:



| SMART MONITORING |
| |
| Hello Inspector 🙋 |
| |
| ● HIGH PRIORITY |
| |
| ABC Foundation |
| Mumbai |
| |
| [START INSPECTION] |
| |
| 2 Other Inspections |
|_____|

It behaves like a mobile application.

12. Inspection Assignment

The government official can click:

Assign Inspection

The backend finds:

- Available inspectors
- Project location
- Inspector location
- Inspector workload
- Project risk
- Previous inspection history

Then assigns someone.

13. Assignment Algorithm

Start simple.

1. Get pending inspections
2. Get available inspectors

3. Calculate distance
4. Check inspector workload
5. Check project priority
6. Select best inspector
7. Create assignment

Example:

ABC Foundation



HIGH RISK



Find inspectors



Inspector 23 → 4 km

Inspector 12 → 25 km

Inspector 07 → 11 km



Inspector 23 selected

Later you can make this an optimization/AI system.

14. Surprise Inspection

The system can randomly select a project.

SURPRISE INSPECTION

Project:

ABC Foundation

Risk:

HIGH ●

Inspector:

INS-023

Status:

ASSIGNED

This directly addresses the surprise inspection requirement.

15. Inspector Starts Inspection

Inspector opens:

START INSPECTION

The browser asks for location permission.

Use:

Browser Geolocation API

Then send:

latitude

longitude

timestamp

to Django.

16. GPS Verification

Your backend knows the project's location.

For example:

Project location:

19.0760, 72.8777

Inspector location:

19.0763, 72.8779

Calculate distance.

If within a defined radius:

Distance: 45 metres

 **LOCATION VERIFIED**

Otherwise:

Distance: 8.2 km

 INSPECTOR NOT AT PROJECT

This is a very strong demo feature.

17. Inspection Checklist

After GPS verification:

INSPECTION CHECKLIST

- ☐ Project operational
- ☐ Staff present
- ☐ Beneficiaries present
- ☐ CCTV working
- ☐ Activities conducted
- ☐ Infrastructure available
- ☐ Records maintained

Store each response in PostgreSQL.

18. Evidence Capture

Inspector can use the browser's camera/file functionality.

Buttons:

[ TAKE PHOTO]

[ RECORD VIDEO]

[ UPLOAD DOCUMENT]

For the prototype, mobile browser camera upload is enough.

19. Evidence Model

Evidence

id

inspection

uploaded_by

file

file_type

latitude

longitude

timestamp

description

So every photo is associated with:

Photo

+

Inspector

+

Inspection

+

GPS

+

Timestamp

20. Inspection Report

At the end:

INSPECTION REPORT

Project:

ABC Foundation

Inspector:

INS-023

Location:

Verified 

Staff:

8 / 10

Beneficiaries:

52 / 100

CCTV:

ONLINE

Issues:

Attendance mismatch

Evidence:

5 Photos

2 Videos

AI Risk:

HIGH 

[SUBMIT REPORT]

21. Attendance Module

Create:

Attendance

id

project

date

reported_count

staff_present

beneficiary_present

created_at

Example:

30 Aug

Reported attendance = 100

Staff = 8

22. AI Attendance Analysis

Now compare data.

Suppose:

Reported attendance = 100

CCTV detected = 43

Your system calculates:

Mismatch = 57

Then:

 ATTENDANCE ANOMALY

23. AI Anomaly Detection

Start with a simple ML model.

Use:

Scikit-learn Isolation Forest

Features:

attendance

staff_count

beneficiary_count

cctv_people_count

cctv_uptime

inspection_frequency

Output:

NORMAL

or

ANOMALY

24. Risk Engine

Create a risk score.

Example:

Attendance mismatch +25

CCTV offline +20

Failed inspection +30

Unusual attendance +15

Repeated issues +10

100

Result:

0–30  LOW

31–60  MEDIUM

61–100  HIGH

25. AI Alert

Create an AIAAlert model.

AIAAlert

id

project

alert_type

description

risk_score

severity

created_at

status

Example:

 **AI ALERT**

Project:

ABC Foundation

Problem:

Attendance mismatch

Reported:

100

Detected:

42

Risk:

82/100

HIGH

26. CCTV — Keep it simple initially

This is where you should not waste your time.

Prototype version:

Use a laptop webcam.

Laptop Webcam



OpenCV



Live stream



Django/streaming layer



React Dashboard

Your dashboard:

ABC Foundation

Main Hall

 LIVE



Camera: ONLINE

27. Real CCTV later

If you have an IP camera:

IP CCTV



RTSP



MediaMTX



WebRTC / HLS



React

This is the more realistic architecture.

But do not make this your first task.

28. CCTV Status

Your database can store:

CCTV

id

project

camera_name

stream_url

status

last_online

Dashboard:

Camera 1  ONLINE

Camera 2  ONLINE

Camera 3  OFFLINE

If camera hasn't responded for a certain period:

 CCTV OFFLINE

29. Computer Vision

After live CCTV works, add:

OpenCV + YOLO

YOLO detects people:

CCTV

↓

YOLO

↓

Person Detection

↓

43 people

Then:

Reported = 100

Detected = 43

⚠ MISMATCH

This creates a very good AI demonstration.

30. Random Video Conference

Use WebRTC.

Official:

[RANDOM VC]

System:

Randomly select project



ABC Foundation



Call Project Incharge

Video:



For your prototype, basic one-to-one WebRTC is enough.

31. Real-Time Updates

Use:

Django Channels + WebSockets

Example:

Inspector submits report.

Inspector



Django



WebSocket



Dashboard

Dashboard immediately shows:

 Inspection completed

No page refresh required.

32. Celery + Redis

Use this for background work.

For example:

Inspection submitted



Celery task



AI analysis



Risk score



Alert

Other tasks:

- Generate reports
 - Process CCTV
 - Send notifications
 - Run AI analysis
-

33. Reports

Create automatic reports.

Possible report types:

Project Report

Project information

Attendance

CCTV

Inspections

Risk

Inspection Report

Inspector

GPS

Checklist

Photos

Videos

Remarks

AI Risk Report

Risk score

Detected anomalies

Historical trends

34. Notifications

For your prototype, start with dashboard notifications.

Example:

 **New inspection assigned**

 **High-risk project detected**

 **CCTV offline**

 **Inspection completed**

 **Attendance anomaly detected**

You can add Firebase later if required.

35. REST API Design

Your Django APIs could be:

AUTH

/api/auth/login/

/api/auth/register/

/api/auth/profile/

PROJECTS

/api/projects/

/api/projects/{id}/

INSPECTIONS

/api/inspections/

/api/inspections/{id}/

ASSIGNMENTS

/api/assignments/

/api/assignments/auto-assign/

ATTENDANCE

/api/attendance/

/api/attendance/{id}/

CCTV

/api/cctv/

/api/cctv/{id}/status/

/api/cctv/{id}/stream/

EVIDENCE

/api/evidence/

/api/evidence/upload/

AI

/api/analytics/

/api/analytics/risk/

/api/analytics/anomalies/

ALERTS

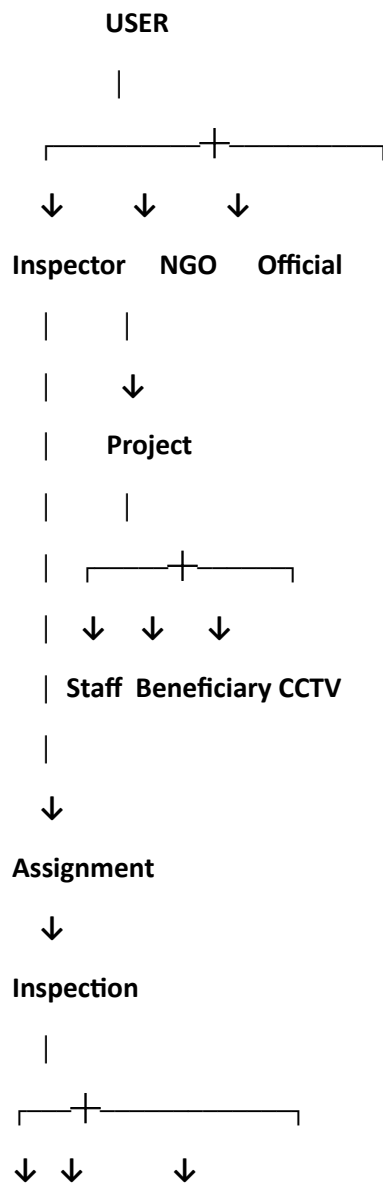
/api/alerts/

REPORTS

/api/reports/

36. Database Structure

Your main relationships:



GPS Checklist Evidence

|

↓

AI Analysis

↓

Risk Score

↓

Alert

37. Security

Implement:

JWT authentication

Role-based permissions

API permissions

Password hashing

HTTPS in deployment

File validation

Don't allow arbitrary dangerous file uploads.

38. Development phases

Now the most important part.

● Phase 1 — Django setup

Build:

- Python environment
- Django
- PostgreSQL
- Django REST Framework
- Custom User
- JWT
- Django Admin

Target: 100% working login.

● Phase 2 — Project management

Build:

- NGOs
- Projects
- Staff
- Beneficiaries
- Locations

Target: Admin can create projects.

● Phase 3 — React dashboard

Build:

- Login
- Dashboard
- Project list
- Project details
- Statistics
- Map

Target: Official can monitor projects.

● Phase 4 — Inspection system

Build:

- Inspection model
- Inspector
- Assignment
- Checklist
- Inspection status

Target:

Official assigns



Inspector receives



Inspector completes

● Phase 5 — Mobile-responsive inspector interface

Build:

- Inspector login
- Home
- Assignments
- Inspection details
- GPS
- Checklist
- Camera upload
- Evidence
- Submit

Target: Open the same React website on your phone and use it like an app.

No React Native.

● Phase 6 — Evidence verification

Add:

- GPS
 - Timestamp
 - Photos
 - Videos
 - Inspector ID
-

● Phase 7 — CCTV

First:

Webcam → Stream → Dashboard

Then, if time:

IP CCTV → RTSP → MediaMTX → WebRTC → Dashboard

● Phase 8 — Attendance

Add:

- Daily attendance
 - Staff attendance
 - Beneficiary attendance
 - Historical graphs
-

● Phase 9 — AI

Start:

Attendance data



Anomaly detection



Risk score



Alert

Then add:

CCTV



YOLO



People count



Attendance comparison

● Phase 10 — WebRTC

Add random video conferencing.

● Phase 11 — Final dashboard

Add:

- AI alerts

- CCTV
 - Map
 - Inspection statistics
 - Attendance graphs
 - Risk scores
 - Reports
-

39. What NOT to build right now

Since you said you don't have much time, don't spend your first days on:

- ✗ React Native
- ✗ Android Studio
- ✗ iOS development
- ✗ Complex cloud infrastructure
- ✗ Advanced deep learning
- ✗ Real government CCTV integration
- ✗ Huge-scale deployment

These are unnecessary for your college prototype.

40. What your final prototype should demonstrate

Your demo should follow one complete story:

1. Government Official Login



2. Dashboard



3. Sees 12 High-Risk Projects



4. Opens ABC Foundation



5. Sees LIVE CCTV



6. AI detects attendance mismatch



7. Risk Score = 82 ●



8. Click "Surprise Inspection"



9. System automatically assigns Inspector



10. Inspector opens mobile-responsive site



11. GPS verification



12. Inspection checklist



13. Takes photo/video



14. Evidence gets GPS + timestamp



15. Inspector submits



16. Dashboard updates

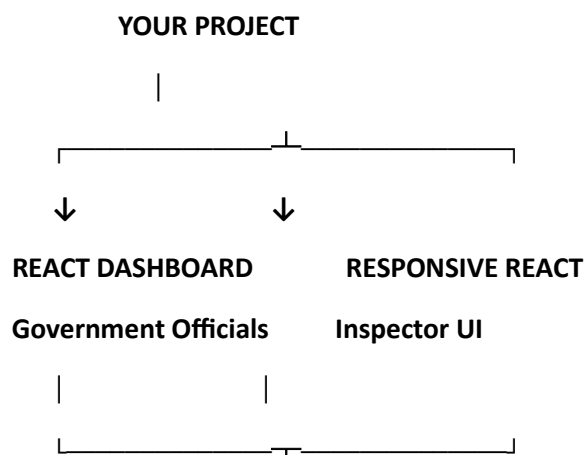


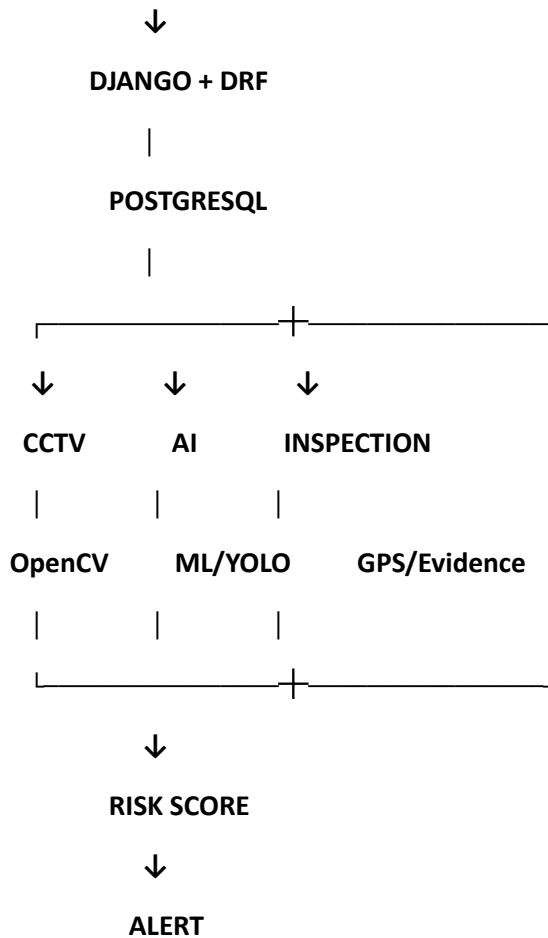
17. AI generates final risk report

🔥 This is the version I recommend you build

You don't need a separate mobile application.

Your stack becomes:





This is much more realistic for your available time. You will still be able to demonstrate the "mobile inspection" requirement because the inspector UI will be designed specifically for a phone screen; you're simply implementing it as a responsive web application instead of a separate Android/iOS application.

so now give a techstach info for the same

For the Smart Real-Time Monitoring & Inspection System – Problem Statement 26095, with no separate mobile application, I recommend this tech stack:

Tech Stack

Layer	Technology	Purpose
Frontend	React.js	Government dashboard + inspector interface
UI	HTML5, CSS3, Tailwind CSS	Responsive/mobile-friendly design
Backend	Django	Main application logic
API	Django REST Framework	Communication between React and Django
Database	PostgreSQL	Store users, projects, inspections, attendance, etc.

Layer	Technology	Purpose
Authentication	JWT	Secure login and API authentication
Maps/GPS	Leaflet + OpenStreetMap	Project locations and geo-tagging
CCTV	OpenCV	Camera/video processing
CCTV Streaming	WebRTC / HLS	Display live CCTV in browser
IP Camera	RTSP	Receive real CCTV streams
Media Server	MediaMTX	Convert/relay CCTV streams for browser
AI/ML	Python, Scikit-learn	Anomaly and risk detection
Computer Vision	YOLO + OpenCV	Detect/count people in CCTV
Real-time Updates	Django Channels + WebSockets	Live dashboard updates
Background Tasks	Celery	AI processing, reports, etc.
Message Broker	Redis	Supports Celery and real-time tasks
Video Conference	WebRTC	Random VC between official and project staff
File Storage	Django Media Storage	Photos, videos, documents
API Testing	Postman	Test Django APIs
Database Management	pgAdmin	Manage PostgreSQL
Version Control	Git + GitHub	Team collaboration
Code Editor	VS Code	Development
Deployment	Render + Vercel	Host backend + frontend

🧩 What each technology does

1. Django — 🧠 Main Backend

Django is the core of your system.

It manages:

Users

Projects

NGOs

Inspectors

Inspections

Attendance

CCTV

Evidence

AI alerts

Reports

2. React.js — Frontend

React creates your interfaces.

You'll have:

Government Dashboard

Dashboard

Projects

CCTV

Map

Inspections

AI Alerts

Reports

and a mobile-responsive Inspector Interface:

My Inspections

↓

Start Inspection

↓

GPS

↓

Checklist

↓

Photo/Video

↓

Submit

You don't need React Native.

3. PostgreSQL — Database

Stores all your structured data.

Users

Projects

Staff

Beneficiaries

Inspections

Attendance

CCTV cameras

Evidence

Risk scores

Alerts

Reports

4. Django REST Framework — Communication

React doesn't directly access PostgreSQL.

Instead:

React



REST API



Django



PostgreSQL

For example:

GET /api/projects/

returns project information to React.

5. JWT — Authentication

Handles login.

Inspector



Login



Django



JWT Token



Inspector accesses dashboard

You can also implement role-based permissions.

6. Leaflet + OpenStreetMap — Location

Used for your project map.

Example:

 Project A

 Project B

 Project C

The inspector's browser can also provide GPS coordinates.

7. OpenCV — CCTV

OpenCV handles video processing.

For your prototype:

Webcam



OpenCV



Video stream



Dashboard

Later:

IP CCTV



RTSP



MediaMTX



WebRTC/HLS



React

8. MediaMTX — CCTV Streaming

This is useful when you move from a simple webcam prototype to actual IP cameras.

CCTV



RTSP



MediaMTX



WebRTC



Browser

Don't make this your first feature.

9. YOLO — Person Detection

YOLO can detect people in the CCTV feed.

Example:

CCTV



YOLO



43 people detected

Then your system can compare:

Reported attendance = 100

CCTV detected = 43



⚠ Attendance anomaly

10. Scikit-learn — 🤖 AI Anomaly Detection

Use it to detect unusual patterns in project data.

Inputs:

Attendance

CCTV count

Inspection history

Staff count

Beneficiary count

CCTV uptime

Output:

Normal

or

Anomaly

Then calculate:

Risk Score = 82/100 ●

11. Django Channels + WebSockets — ⚡ Real-Time

Suppose inspector submits an inspection.

Without WebSockets:

Inspector submits



Official refreshes page



New data appears

With WebSockets:

Inspector submits



Django



WebSocket



Dashboard updates instantly

This helps fulfill the real-time monitoring aspect.

12. Celery + Redis — Background Processing

Use these for tasks that shouldn't slow down Django.

For example:

Inspection submitted



Celery



AI analysis



Risk score



Alert

Also useful for:

- Report generation
 - CCTV processing
 - Scheduled analytics
 - Notifications
-

13. WebRTC — Video Conference

For random VC:

Government Official



WebRTC



Project Incharge

This can be added after your core inspection system is working.

14. Django Media Storage — Evidence

Inspector uploads:

 Photos

 Videos

 Documents

Django stores the file and database stores its information:

Evidence

├— Inspection ID

├— Inspector

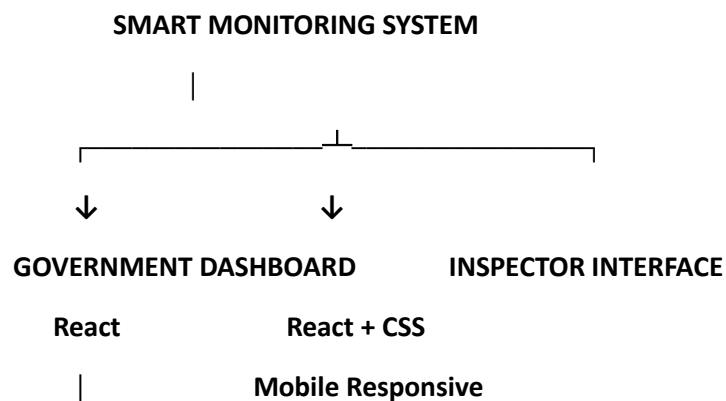
├— GPS

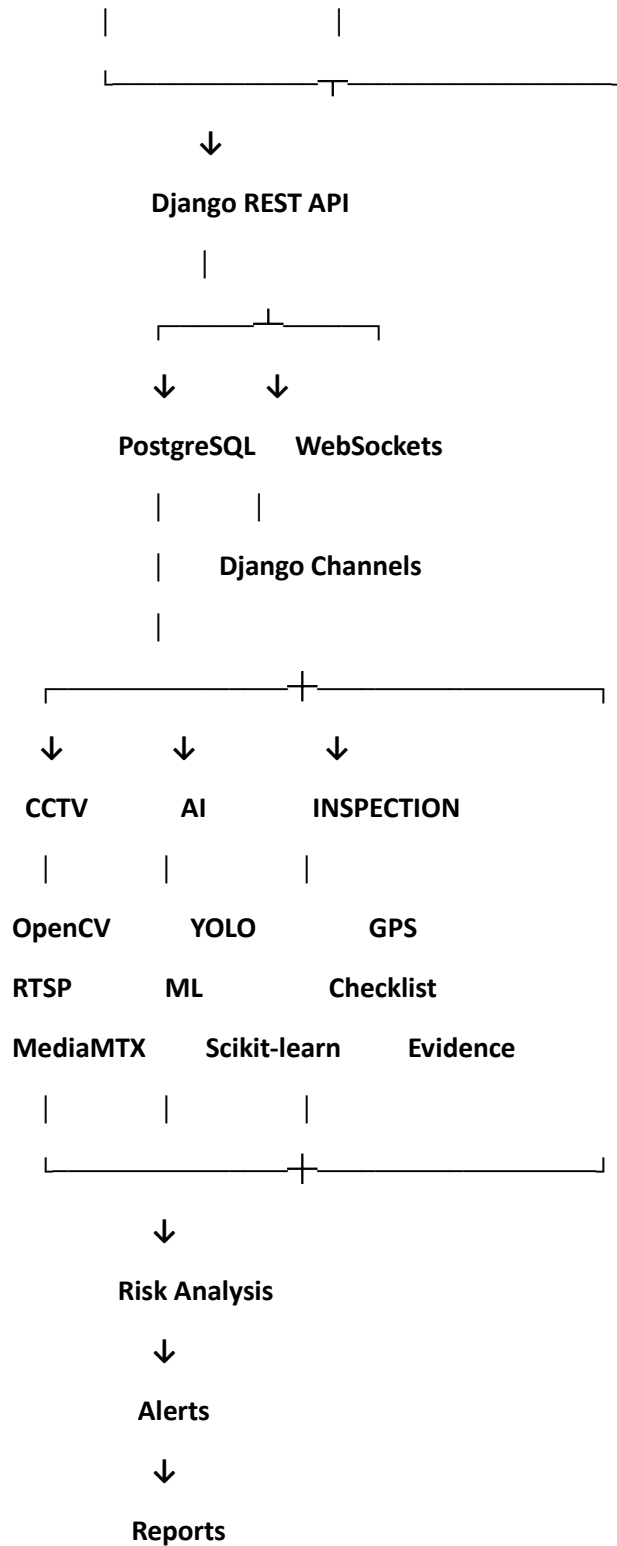
├— Timestamp

└— File

For your college prototype, this is enough. You don't need AWS S3 immediately.

Final Architecture





★ For your limited time

You do not need to implement every technology.

● Essential

Python

Django

Django REST Framework

PostgreSQL

React

HTML/CSS

Leaflet

Git/GitHub

Postman

 **Add after the core works**

OpenCV

CCTV streaming

AI anomaly detection

Django Channels

Redis

Celery

 **Advanced/demo enhancements**

YOLO

WebRTC

RTSP

MediaMTX

Cloud deployment

So your core stack can simply be:

React + Django + DRF + PostgreSQL + Leaflet + OpenCV + Scikit-learn